Relatório Redes EP 1 - Club Penguin

Vinicius Chirnev - 15580531

Lucas Giovani Santos Ross - 15471693

Tiago Silveira Almeida - 15490509

1. Proposta

A ideia original do nosso trabalho era criar uma versão alternativa do Club Penguin inicial, no qual apenas existia uma tela branca com alguns pinguins que tinham username, podiam dançar e andar pela tela através do clique do mouse, além de enviar mensagens via chat. Porém, na nossa versão os pinguins seriam substituídos por cobras, isso em referência a um grupo de colegas da faculdade que chama-se Voltantes, os quais possuem como ícone uma cobra.

Imagem 1: Versão inicial do Club Penguin

2. Especificação

2.1. Fundamentos e Arquitetura

Para a construção da nossa aplicação utilizamos Java e suas bibliotecas para comunicação em rede, como a java.net e java.io, além de suas bibliotecas para construção de interface gráfica, como javax.swing e java.awt, por fim usamos também threads e listas bloqueantes para controle de fluxo que serão explicadas mais à frente.

2.1.1. Modelo de Comunicação

O modelo de comunicação que escolhemos foi de Cliente-Servidor, uma arquitetura centralizada, isso porque, dado nosso projeto, gostaríamos que a aplicação pudesse sempre receber novos clientes, assim como no Club Penguin, poderiam existir servers vazios. Além disso, seria bom manter o controle de quem estaria entrando e saindo da aplicação dado o fato de termos implementado um sistema de login, logo teríamos que salvar as posições antigas em que o cliente estava, sua senha e seu username, para que ele pudesse entrar novamente, e isso seria feito no servidor.

2.1.2. Camada de Transporte

O protocolo de mensagens utilizado foi o TCP, isso porque as mensagens trocadas precisavam ser garantidamente enviadas e recebidas em ordem para o gerenciamento dos estados, além disso enviamos mensagens de texto via chat, essas mensagens precisam ser enviadas inteiras para manter seu sentido e o TCP nos garante isso.

Dado o grande número de atualizações de estado e dado a quantidade de jogadores se movimentando e fazendo ações foi necessário tratar cada conexão individualmente para garantir que não ocorressem movimentações estranhas na tela, ou seja, garantir a integridade do jogo.

2.1.3. Mecanismos de E/S

O envio e recebimento de mensagens foi feito por meio das classes ObjectOutputStream e ObjectInputStream principalmente porque o servidor enviaria o Map com os estados de todos os jogadores e os clientes enviariam ao servidor suas coordenadas x e y que estão sendo tratadas como uma classe Ponto2D. Por este motivo, todas as nossas mensagens são objetos serializáveis que diferem em seu conteúdo, nesse ponto o Java nos fornece uma ferramenta muito agradável já que as referidas classes de E/S já são bufferizadas e garantem melhor performance do código.

Essas classes foram utilizadas tanto em Cliente quanto em ClientHandler para fazer o controle de todas as E/S, instanciamos ambos OOS e OIS fazendo referência ao socket já com a conexão estabelecida, e especificamente em cliente passamos essas instâncias para o mensageiro do cliente e para o leitor do cliente, controlando as mensagens enviadas através de métodos chamados pela interface gráfica e as mensagens recebidas através de uma thread que aguarda mensagens do servidor, respectivamente.

2.2. Estrutura de Mensagens

Esta seção descreve o formato dos dados trocados entre servidor e cliente.

2.2.1. Classe Base e delimitação

Todas as nossas mensagens são criadas a partir de uma herança da classe base que implementa Serializable (uma interface que permite transformar um objeto em uma sequência de Bytes para podermos ser capazes de realizar a comunicação entre sockets) e possuem como delineadores de campo seus atributos.

O benefício dessa classe é que podemos garantir que todos os objetos passados apenas poderão ser do tipo “MensagemBase” ou alguma de suas extensões para outro socket, permitindo a identificação do tipo da mensagem por meio de um switch/case em que a verificamos o tipo da mensagem pelo resultado de um instanceof.

2.2.2. Tipos de Mensagens de Fluxo e Controle

MsgLogin:

Tipo: Requisição
Sintaxe: username e senha
Semântica: username é uma string que representa o nome escolhido para a conta e a cobra do jogador, e a senha é a palavra-passe utilizada para entrar em sua conta.
Regras: Essa mensagem é enviada assim que o Cliente se conecta ao servidor, por meio da inserção dos seus dados de usuário e senha pelo cliente, como resposta o ClientHandler confere as informações, e manda uma mensagem resposta dizendo se o cliente foi capaz de logar ou criar uma conta.

MsgRespostaLogin:

Tipo: Resposta
Sintaxe: mensagem, id, login
Semântica: mensagem é uma string que informa ao jogador se ele conseguiu entrar no servidor "ITS SNAKE TIME!!!", ou se o jogador já está conectado "COBRA JA CONECTADO! FECHANDO CONEXAO...", ou ainda se o usuário erra a senha "SENHA INCORRETA! VOCE ESTA SENDO DESCONECTADO...". O id é um int passado ao cliente referente a sua cobra, utilizado para renderizar o seu personagem. Já o login é um booleano que se for True permite o login.
Regras: Essa mensagem é enviada como uma resposta a requisição do login do cliente, e define os valores de seus parâmetros no ClientHandler, e como consequência da resposta, é permitir (ou não) o login do cliente e informá-lo da situação.

MsgFluxo:

Tipo: Requisição e resposta
Sintaxe: idremovido, username, tipo

Semântica: tipo é uma String que indica se o fluxo é de entrada ou saída, username é uma String que indica a conta do usuário e idremovido é um inteiro utilizado para parar de renderizar a cobra do player.

Regras: A mensagem fluxo pode ocorrer em dois momentos diferentes, quando o player entra no servidor com o seu login validado o ClientHandler manda a MsgFluxo com o tipo "entrada" avisando todos os outros player que ele entrou no server. Mas ela também pode ocorrer quando um player sai do servidor por meio do método closegeral que envia a MsgFluxo com o tipo "saída" que tem como resposta um aviso para todos os outros players que aquela cobra saiu do servidor.

MsgposiçãoAlvo:

Tipo: Requisição

Sintaxe: Ponto 2D

Semântica: Ponto2D é uma classe que possui dois inteiro X e Y utilizados para representar a posição das cobras.

Regras: Essa mensagem pode ser enviada quando o cliente já realizou login na interface, por meio de um clique na tela, fazendo com que seja solicitado uma atualização da posição da cobra ao servidor.

MsgChat:

Tipo: Requisição

Sintaxe: username e texto

Semântica: username é uma String que representa o nome do usuário que deseja enviar a mensagem e texto é outra String que possui o conteúdo digitado pelo usuário.

Regras: Essa mensagem pode ser enviada quando o cliente tem o seu login aceito, e ocorre quando o player envia uma mensagem pela caixa de texto, fazendo com que seja solicitado para o server enviar essa mensagem para todos os outros usuários online.

MsgDança:

Tipo: Requisição

Sintaxe: Essa mensagem não possui atributos (basta o tipo da sua classe para ser identificada a solicitação de dança)

Semântica:

Regras: Essa mensagem pode ser enviada quando o cliente tem o seu login aceito, e ocorre quando o player clica no botão de dança, pedindo para atualizar o estado da cobra no servidor (ligando ou desligando), fazendo com que outros player vejam a animação de dança começar/parar.

MsgAtualizacaoEstado:

Tipo: Resposta

Sintaxe: MAP<Integer, Cobra> estadoGlobal

Semântica: o Map estadoGlobal relaciona os ID's dos jogadores com as suas cobras, ele é utilizado para atualizar toda a mudança de estado que tenham efeito sobre as cobras.

Regras: O atualiza estado só pode ser executado depois que o ClientHandler recebe algum tipo de mensagem, como por exemplo uma solicitação de login, saída, movimentação ou dança. Após realizar o tratamento da solicitação envia aos jogadores os novos estados das cobras permitindo a "visualização do estado real delas".

2.3. Fluxo de Comunicação e Inicialização

2.3.1. Estabelecimento de Conexão e Login

Fluxo de Login: O cliente envia MsgLogin, e o servidor verifica o bancoDeContas sob um bloco synchronized, caso encontre a conta do usuário no banco verifica se ele está online, caso esteja significa que esta conta já está sendo usada, portanto manda uma nova mensagem ao cliente do tipo MsgRespostaLogin, avisando que a conta escolhida já está conectada e dentro dessa mensagem manda um booleano com o valor False impedindo o login do cliente.

Se o cliente não estiver online, confere se a senha enviada pela MsgLogin está correta com a senha referente a conta no Banco de contas, caso não esteja correta manda uma MsgRespostaLogin informando que a senha enviada está errada e impede o login do cliente, mas caso a senha enviada pelo cliente esteja correta, marca o atributo online de conta o valor True, e atribui um ID para ela com base no index do banco de contas, além de mandar uma mensagem permitindo e informando o sucesso no login para o cliente.

No caso de não encontrar nenhuma conta referente ao nome de usuário enviado pelo cliente, é criada uma nova conta e em seguida adiciona no bancoDeContas, e manda uma mensagem do tipo MsgResLogin informando o usuário que uma nova conta foi criada e permite o seu login no servidor.

2.3.2. Pós-login e Ciclo de Vida geral da aplicação

O login é a primeira e única barreira para que a parte principal do projeto - o jogo - seja iniciada. Após o login, o ClientHandler faz alguns processos no lado de servidor para preparar a entrada do jogador entrante e, logo após isso, envia a primeira mensagem MsgAtualizacaoEstado (o estado global inicial) apenas para o cliente que acabou de se conectar, permitindo que esse inicie seus próprios processos internos relacionados, especialmente, à interface gráfica. Esta é a única atualização de estados Unicast feita no jogo, as próximas que este cliente receber serão todas atualizações pelo Broadcast.

Após isso, o ClientHandler do cliente entrante faz 2 broadcasts:

1. Atualização de estados (MsgAtualizacaoEstado) garantindo que todos os jogadores possam ver o jogador entrante;

2. Mensagem de entrada (MsgFluxo) garantindo que todos os jogadores recebam uma mensagem no Chat indicando a entrada do novo jogador;

Com isso, a partir de agora tanto Servidor quanto Cliente terão seus objetos e/ou métodos de escuta e envio que ficarão ativos por meio de multithread - como o próprio ClientHander para o Servidor e MensageiroCliente, dentre outros, para o Cliente (especificações de cada classe mais a frente). Desse modo, o funcionamento da aplicação se resume em Cliente poder enviar requisições de ação dos tipos MsgChat, MsgDanca, MsgPosicaoAlvo a qualquer momento (dependendo do input do usuário) e Servidor esperar essas requisições para:

1. Mudar os estados globais dos jogadores e realizar um broadcast das atualizações por meio da mensagem MsgAtualizacaoEstado para todos os outros clientes conseguirem manter a consistência visual;
2. Ou realizar um broadcast das mensagens de Chat de outros jogadores por meio da mensagem MsgChat para todos conseguirem acompanhar o Chat do jogo em tempo real.

2.3.3. Objetos auxiliares de Cliente

Como mencionado no tópico anterior, Cliente utiliza técnicas de multithreads para conseguir:

1. Receber mensagens;
2. Enviar mensagens;
3. Gerenciar os estados locais dos jogadores (informações utilizadas pela GUI);
4. Desenhar a GUI e controlar inputs do jogador (há duas classes que executam essas funções);

Nesse tópico discorreremos sobre as primeiras 2 funcionalidades, pois são as que envolvem comunicação com o Servidor.

Sendo assim, Cliente possui uma thread de leitura dedicada por meio da classe LeitorDoCliente que possui um loop que bloqueia no ois.readObject() e garante que o Cliente sempre esteja podendo receber dados de servidor. Logo, todas as mensagens que Cliente recebe são intermediadas por LeitorDoCliente, que funciona como um decodificador, recebendo uma mensagem em forma de Object, devido à serialização, convertendo-a em MensagemBase e checando de que classe é essa mensagem por meio do *instanceof*. Ao identificar a classe, a mensagem é convertida para a classe correta (MsgAtualizacaoEstado, MsgChat ou MsgFluxo) e LeitorDoCliente delega a próxima etapa para outro objeto - como GerenciadorDeEstados ou Jogo.

Continuando, Cliente também possui uma thread dedicada ao envio de mensagens por meio da classe MensageiroCliente. Essa classe pode ser chamada por

PainelJogo e Jogo - classes responsáveis pela GUI e pelos inputs - e, em suma, é quem vai enviar todas as mensagens advindas de input para o Servidor. Logo, MensageiroCliente faz requisições ao Servidor enviando MsgDanca, MsgFluxo, MsgPosicaoAlvo ou MsgChat.

Por fim, cabe mencionar a classe GerenciadorDeEstados - responsável por usar MsgAtualizacaoEstado recebida do servidor para atualizar os estados locais do Cliente - que, apesar de não enviar ou receber mensagens diretamente, utiliza uma estrutura de BlockingQueue (filaDeEventos) e uma thread processadora para garantir que:

1. LeitorDoCliente, ao receber uma MsgAtualizacaoEstado, não tenha que esperar GerenciadorDeEstados realizar seu método até o fim antes de voltar a receber mensagens, podendo delegar essa tarefa completamente e voltar a fazer sua função;
2. Não desperdiçar CPU enquanto a filaDeEventos estiver vazia. Algo que aconteceria se utilizássemos uma fila normal em um loop.

Sendo assim, GerenciadorDeEstados funciona de modo a permitir que LeitorDoCliente possa executar exclusivamente sua tarefa de decodificar mensagens e repassá-las.

Similarmente, Jogo também tem um mecanismo que gera o mesmo resultado que GerenciadorDeEstados por meio da função *SwingUtilities.invokeLater()* da biblioteca de interface gráfica.

2.4. Estabilidade e Manutenção da Conexão

2.4.1. Controle de Concorrência no Servidor

O controle de concorrência foi não só uma das grandes dificuldades do projeto (que será ressaltada na seção de testes do relatório) mas também um dos pilares mais essenciais para o funcionamento adequado da aplicação. Essa seção tratará das estruturas Server-side, mas tudo dito aqui também se aplica para estruturas Client-side.

O problema surge do fato de que possuímos algumas estruturas de dados centrais para o funcionamento de todo o Jogo - dentre elas, *estadosGlobais* (Server-side), *bancoDeContas* (Server-side), *clientHandlerList* (Server-side) e *estadosGlobaisCliente* (Client-side) - e, devido a diferentes threads de clientHandler funcionando em paralelo, frequentemente há situações em que diferentes clientHandlers tentam acessar os mesmo dados, gerando uma condição de corrida. Um exemplo é quando um cliente envia uma requisição MsgPosicaoAlvo e, enquanto as informações de estadosGlobal são atualizadas no Servidor, outro cliente envia uma requisição MsgPosicaoAlvo. Sem tratamento, a thread de um clientHandler tentaria acessar o HashMap estadosGlobal enquanto a thread do outro clientHandler já está modificando a mesma estrutura. Isso poderia gerar ConcurrentModificationException.

Algo similar acontece com as funções broadcast() e removeClientHandler() - requisitadas por mensagens variadas -, dado que ambas utilizam clientHandlerList. Consequentemente, se uma thread tenta iterar a lista para fazer um broadcast e outra tenta remover um clientHandler da lista, temos novamente uma condição de corrida.

De modo geral, isso pode acontecer em diversas combinações diferentes de métodos e muitas vezes a consequência é um Exception em clientHandler que interrompe seu funcionamento e, por conseguinte, fecha a conexão com um Cliente imprevisivelmente.

A correção desses problemas foi possível por meio de "locks" na aplicação. Ou seja, utilizando a keyword *synchronized* do java para proteger sequências de operações, como Get → Modify → Broadcast, por exemplo. Ou seja, quase em todas as circunstâncias em que um método acessa estadosGlobais, bancoDeContas ou clientHandlersList, um synchronized, garantindo que não hajam problemas de concorrências entre threads.

2.4.2. Keep-Alive e Detecção de Falha

Utilizamos um mecanismo de Keep-Alive na nossa aplicação, no qual o cliente envia para o servidor uma mensagem sem conteúdo a cada 15 segundos, no caso do cliente ficar sem enviar nenhuma mensagem - nem mesmo a de Keep-Alive - por mais de 30 segundos, o servidor desliga a conexão com este cliente. Esse mecanismo foi implementado, pois caso não haja nenhum outro cliente conectado - ou os outros clientes não enviem nenhuma mensagem - e o único cliente feche a aplicação de forma inesperada, ambos os lados estariam estagnados e a conexão nunca seria fechada corretamente.

Ademais, outros tipos de falha, como uma falha no login ou falhas na transmissão de mensagens (por fechamentos inesperados ou até mesmo ações fora do esperado no cliente) são tratadas com o encerramento da aplicação no lado do cliente e encerramento da conexão e então da thread no lado do servidor.

2.4.3. Detalhes de segurança do método Broadcast()

Esse tópico foi separado para explicar uma peculiaridade do método broadcast() da classe ClientHandler.

Broadcast() é um dos métodos mais importantes da aplicação dado que ele é responsável pela maioria das mensagens enviadas pelo Servidor aos vários Clientes. Consequentemente, ele é acessado com frequência e por diferentes threads. Desse modo, é possível se deparar com o seguinte problema:

Um cliente pode sair de dois modos diferentes:

1. Enviando uma MsgFluxo (saída) e desconectando-se de maneira segura;
2. Perdendo a conexão abruptamente, seja por ter encerrado o processo, perdido conexão à internet ou outro fator inesperado.

Caso um cliente saia do segundo modo, seu clientHandler continuará ativo até que a função de keepAlive() - depois de um tempo - o desative. Se durante esse tempo, outro clientHandler tentar um broadcast, ao iterar na clientHanlersList e tentar enviar uma mensagem para o Cliente que caiu, haverá um Exception devido ao socket fechado do lado do Cliente e, se não houver um bloco try-catch, o clientHandler que está realizando o broadcast será interrompido - gerando, inclusive, um problema de ponteiro vazio na clientHandlersList. Esse problema poderá acontecer diversas vezes enquanto o clientHandler não é finalizado, dado que diversos broadcasts podem ser realizados antes do keepAlive() fechar o clientHandler com problema.

Desse modo, uma lógica de coleta de clientHandlers caídos foi implementada, funcionando por meio de uma lista temporária que adiciona em si todos os clientHandlers que devem ser removidos por meio de um bloco try-catch. Ao final do broadcast, os clientHandlers problemáticos são corretamente finalizados, garantindo o funcionamento correto do código.

2.4.4. Limpeza e Finalização de Sessão

Função do closeGeral(): Método responsável por liberar recursos de forma coordenada: Inicialmente confere se a conta do jogador existe e se a sua conta está online. caso não esteja, apenas fecha o ClientHandler e garante o fechamento do socket e dos streams.

Caso as condições sejam atendidas e há uma cobra na lista de estados globais para salvar as informações antes de sair (que no pior caso pode não encontrar devido a condições de corridas na utilização de Threads), salva a posição da cobra em sua conta para caso o player reconecte apareça na mesma posição, e depois remove a cobra dos estadosGlobais fazendo com que a sua cobra pare de renderizar para outros jogadores, marca a Conta como online=false, e remove o ClientHandler da lista de broadcast e fecha os Sockets.

Também há um tratamento específico de login falho: Uso de closeAlternativo() para fechar rapidamente as conexões que não completaram o login, apenas fechando os ObjectsStreams e o socket em si, já que não foi criado nenhum player para ser necessário a sua remoção, evitando também sobrecarga no closeGeral completo.

Perante essas finalizações de sessão que ocorrem todas no lado do servidor, no método closeGeral() enviamos uma mensagem de fluxo do tipo saída para todos os clientes do servidor por meio de um broadcast, esta mensagem de saída é recebida em cliente e apenas serve para informar da saída de um cliente do servidor (uma mensagem que aparecerá no chat). Como todas as conexões são fechadas no lado do servidor, ao desconectar o cliente envia uma mensagem de fluxo de saída para o servidor que a trata chamando o modo de closeGeral(), isso acontece quando o cliente aperta o botão sair na GUI.

3. O Código

3.1. Lado do cliente

O código é dividido em 3 pastas: cliente, compartilhado e server, na pasta de cliente existe a classe principal - a qual será executada - Cliente.java, nela iniciamos a conexão com o servidor, pedindo para o usuário inserir um IP e uma porta com a qual se conectará. Feita a conexão criamos uma instância de cliente para que possamos usar os métodos dele na main, o cliente então pedirá por um login com o username e senha inseridos pelo usuário, caso o login tenha sucesso, o cliente inicia uma instância de mensageiro que ficará responsável por enviar as mensagens ao servidor, inicia também o seu Map que controlará os estados locais das cobras e um gerenciador de estados que é uma classe a qual atualizará os estados neste Map local, após isso cliente inicia uma thread leitora que ficará esperando mensagens do servidor chegar, com todo o ambiente de troca de mensagens pronto, o jogo (ou a GUI se assim preferir) será iniciado também em uma thread separada (está já implementada na biblioteca swing), desenhando o Map com os estados atualizados em tempo real, por último o cliente inicia sua thread do Keep-Alive.

3.2. Lado do Servidor

No lado do servidor iniciamos pela classe principal Server.java, na sua main pedimos para o usuário inserir a porta na qual abrirá o servidor e então criamos o welcome socket e instanciamos um servidor apenas para usar os métodos dele na main, e então chamamos o método abreServidor() que aguarda algum cliente iniciar conexão, quando uma conexão for feita, uma instância de ClientHandler é criada com o socket do cliente que fez conexão e este clientHandler é colocado para rodar em uma thread separada. No método run() de ClientHandler é que a troca de mensagens com o cliente é tratada. Primeiro tratamos o login no método de mesmo nome, que recebe uma mensagem de login do cliente verificando no banco de contas do servidor se o usuário já existe, caso exista verifica se a senha inserida foi a correta e então atualiza suas variáveis para sincronizar-se com o cliente ao qual está se comunicando, caso a senha passada esteja incorreta finaliza a conexão com esse cliente, caso o usuário passado não exista, cria uma conta com este usuário e adiciona ela a lista de contas do servidor, em ambos os casos onde o login é bem sucedido ou não, envia uma mensagem de resposta login, contendo o id da sua cobra e o estado de seu login.

Feito o login no lado do servidor, adicionamos o novo ClientHandler em uma lista de ClientHandler, a qual basicamente servirá para enviarmos comunicações a

todos os clientes por meio do método de broadcast(), então com todos os meios para a comunicação com o cliente devidamente inicializados, criamos a instância de cobra para o cliente e adicionamos ela ao Map global sendo armazenado no servidor, envia então para o cliente o estado atual do Map e faz um broadcast desse estado a todos os clientes do servidor, juntamente com um broadcast que sinaliza a entrada de um novo jogador no servidor. Estabelecida a entrada do novo cliente ao servidor, o ClientHandler começará a receber as mensagens enviadas pelo cliente o qual é responsável por, isso é feito pelo método keepAlive() e recebeMensagem().

4. Como Compilar

Para compilar o código manualmente via terminal basta viajar até a pasta onde o diretório src está localizado, junto dele deve haver uma pasta bin (para onde irão as classes compiladas) e uma pasta dist (onde estão os executáveis .jar) a partir dessa pasta, que representa a pasta principal do projeto, o comando `javac -sourcepath src/ src/cliente/*.java src/compartilhado/*.java src/compartilhado/mensagens/*.java src/server/*.java -d bin/` deve ser executado, ele irá compilar todas as classes necessárias para o funcionamento do código. Essa forma de compilação via terminal foi testada apenas no Windows 10, por isso não garantimos que irá funcionar em outro SO, mas basta compilar as classes do diretório principal que deve funcionar.

Além disso, como nossa aplicação possuí uma GUI, alguns sprites foram criados, eles se localizam na pasta src/sprites/ porém, a compilação via terminal não conseguirá visualizar os sprites caso eles não estejam na pasta bin, por isso após a compilação das classes você poderá executar o comando `Copy-Item -Path "src\sprites" -Destination "bin" -Recurse` no terminal do Windows, ou o comando `cp -r src/sprites bin/` em Linux/macOS. Ao final da execução destes comandos a nossa aplicação deve poder ser executada.

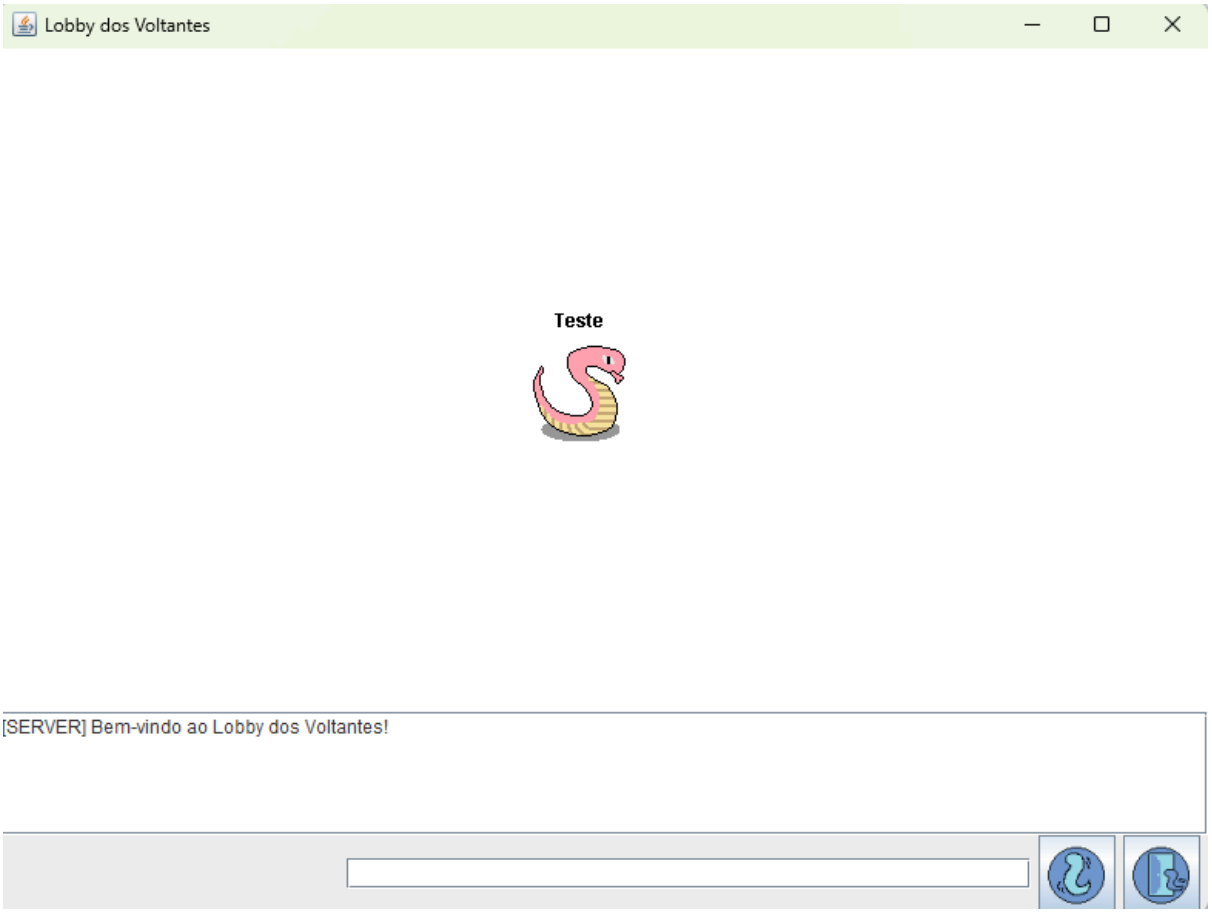
5. Como Executar

Para executar o código compilado via terminal a partir da pasta principal, caso você esteja do lado do servidor deverá executar o código `java -cp bin/ server.Servidor` e então iniciar o servidor normalmente. Caso esteja do lado do cliente você deverá executar o código `java -cp bin/ cliente.Cliente` e então iniciar a conexão com o servidor normalmente.

Caso prefira utilizar o executável basta viajar até a pasta dist/ e dela utilizar o comando de terminal `java -jar ServidorLobbyVoltantes.jar` do lado do servidor, e então iniciar o servidor normalmente ou então `java -jar ClienteLobbyVoltante.jar` do lado do cliente e então iniciar a conexão com o servidor normalmente.

Ambas as formas de execução foram testadas em Windows 10 e 11 com diferentes versões do Java, caso utilize outro sistema operacional é recomendado que opte pela compilação e execução via terminal que se difere pouco de um SO para outro.

6. Interface



A nossa interface consiste em um programa com sprites criados pelo grupo, em que você é capaz de mover,mandar mensagens, dançar e sair do servidor.

Mover: Para andar simplesmente clique com o botão esquerdo do mouse em um espaço em branco em cima do chat, e a sua cobra se moverá para aquela direção.

Conversar: Para mandar uma mensagem clique no retângulo à esquerda dos ícones azuis e escreva o texto que deseja enviar, para mandar apenas pressione “Enter”.

Dançar: Para começar a dançar apenas pressione o seguinte ícone:

Assim que pressioná-lo a cobra iniciará uma dança (tenha em mente que ela não capaz de se mover enquanto está dançando) e para terminar a animação pressione o botão novamente.

Sair do servidor: Para sair do servidor pressione o seguinte ícone:
Ao pressioná-lo, o programa finaliza a sua execução.

7. Testes

7.1 Testes de sobrecarga e condição de corrida

Como mencionado em diferentes tópicos da seção 2, a condição de corrida foi um dos grandes obstáculos do projeto - se não o maior - e, consequentemente, os testes a seguir foram 90% dos testes realizados. Tais testes, de modo geral, envolveram:

1. Entrar no servidor com cada vez mais clientes simultâneos até algo falhar;
2. Com vários clientes diferentes, realizar ações diversas ao mesmo tempo;
3. Gerar erros de login enquanto há usuários logados;
4. Entrar e sair do servidor com mecanismos incorretos (como fechar o processo) enquanto há outros clientes logados.

Estes testes, apesar de diferentes, revelaram/causaram os mesmos tipos de falhas (condição de corrida), mas em diferentes lugares e sob diferentes condições. Como mencionado na seção 2, os principais problemas foram em estadosGlobais, bancoDeJogadores, clientHandlersList e estadosLocais que quando haviam poucos jogadores não causavam problemas mas quando entrava uma quantidade maior, geraram Exceptions por threads diferentes tentarem acessar os mesmos dados.

Ou seja, notamos que muitos jogadores online geravam uma sobrecarga no servidor, no sentido de escancarar os problemas de condição de corrida - quanto mais threads, maior a chance de ocorrerem esses problemas.

Não apenas isso, mas problemas como o do broadcast (citado na seção 2) foi notado ao entrarmos com vários clientes e sair com mecanismos incorretos.

Por fim, um erro no keepAlive() foi notado pois ele não possuía a linha de Thread.sleep(1000), o que gerava uma sobrecarga de busy-waiting em uma das threads de ClientHandler, travando o terminal (ou pelo menos deduzimos

que isso foi uma das causas do terminal travar) de Servidor após poucos Clientes terem entrado.

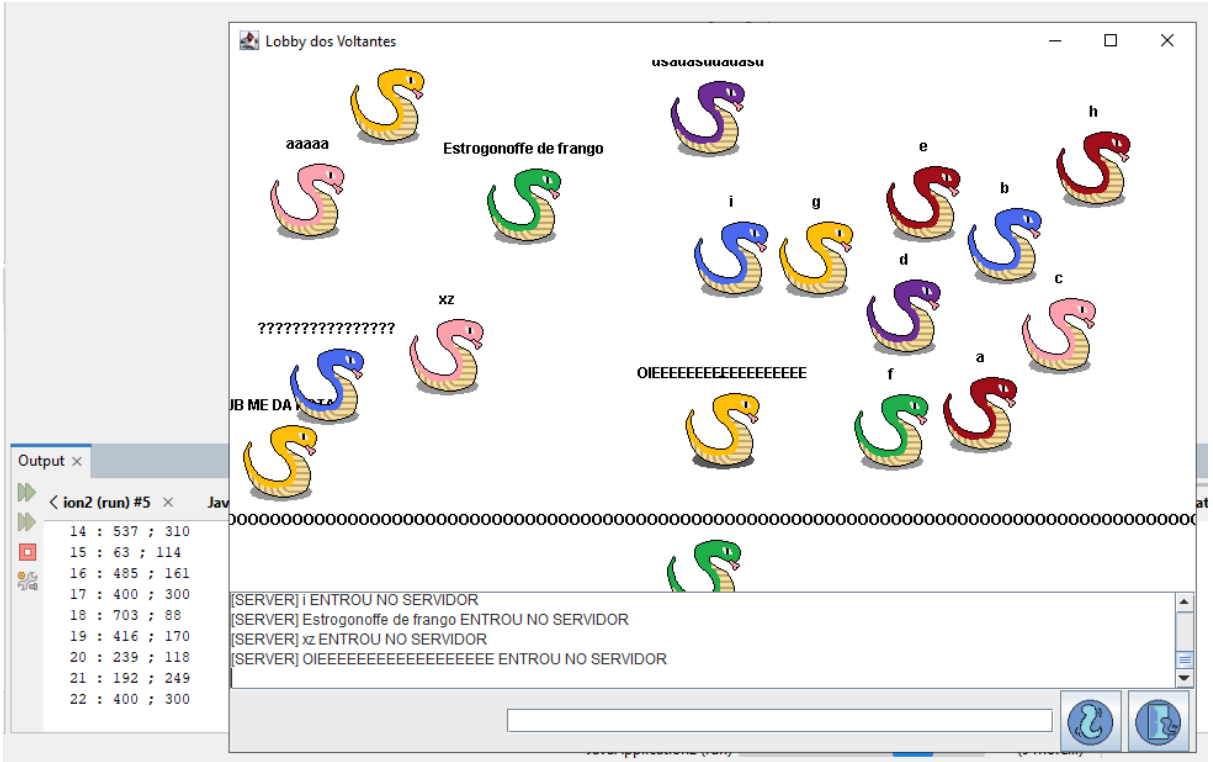


Imagem 2: 22 clientes conectados ao mesmo tempo.\

7.2 Teste de conexão VPN

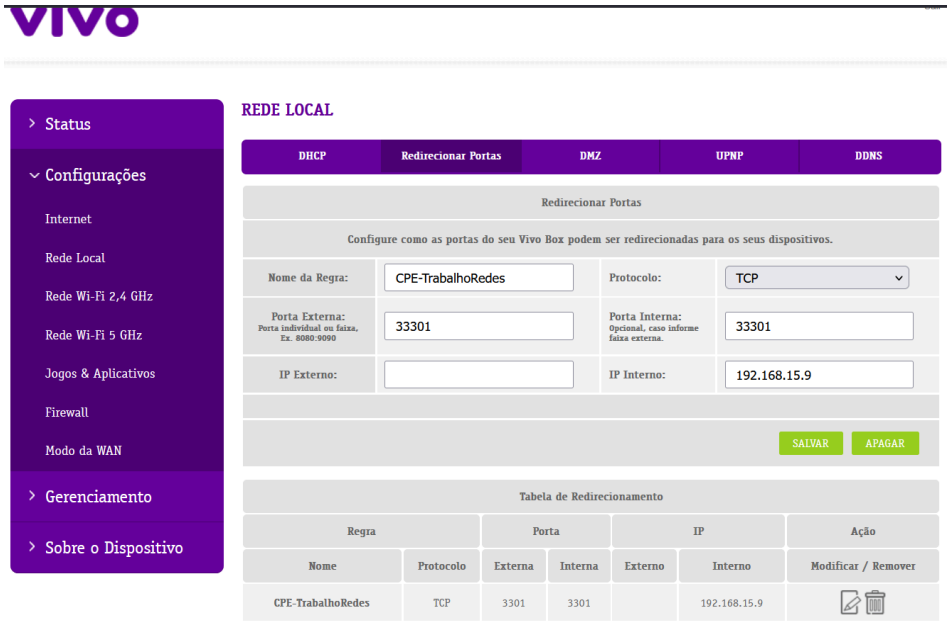
O primeiro teste de conexão entre diferentes máquinas foi feito utilizando o RadminVPN, isso porque ele fornece um IPv4 real ao qual os clientes podem se conectar. O usuário que executou o servidor e os usuários que executaram o cliente se conectaram a mesma rede de VPN e usaram o IP fornecido por ela. Dessa forma nossa aplicação funcionou plenamente, com todas as mensagens sendo enviadas e recebidas de forma integral, nesse teste ambos os lados utilizaram Windows 10 e conseguimos conectar 20 clientes ao mesmo tempo, acreditamos que seria possível adicionar mais, porém não fomos além disso por não achar necessário.

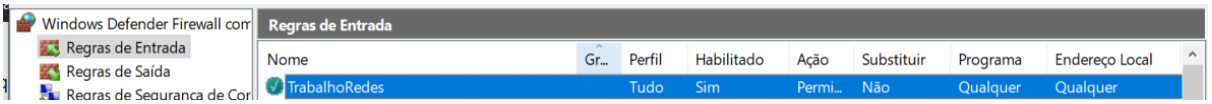
A conexão foi relativamente simples, pois funcionou quase como uma conexão local, apenas fornecendo o IP e a porta, e ambos os lados estando conectados à mesma rede de VPN.

7.3 Teste de conexão IPV4

O segundo teste de conexão entre diferentes máquinas teve como objetivo tentar uma conexão bem sucedida sem o uso de VPNs simulando redes locais. Para isso, tentamos utilizar o IPV4 como “host” na comunicação

de sockets, mas logo notamos que isso não seria tão simples de ser efetuado. O primeiro passo foi tentar realizar “port fowarding” no roteador do computador servidor e adicionar regras de entrada no firewall permitindo requisições externas na porta do socket (33301). No entanto, a conexão não foi possível mesmo após esses fatores e questionamentos foram levantados sobre a possibilidade de um mascaramento de IP CGNAT na rede, o que nos fez tentar fazer a conexão via IPV6.





7.4 Teste de conexão IPV6

A princípio, a conexão via IPv6 também não funcionou, mesmo ao desativar completamente o Firewall do computador Servidor. No entanto, após pesquisas, tentamos criar uma regra de entrada no Firewall do roteador, permitindo requisições externas na porta 33301 no IPv6 da rede do Servidor.

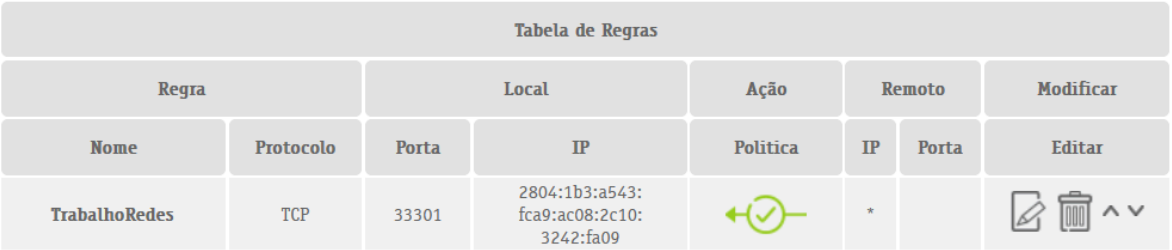


Tabela de Regras

Regra		Local		Ação	Remoto		Modificar
Nome	Protocolo	Porta	IP	Politica	IP	Porta	Editar
TrabalhoRedes	TCP	33301	2804:1b3:a543:fca9:ac08:2c10:3242:fa09		*		

Após isso, a conexão por meio de IPv6 foi um sucesso e computadores diferentes passaram a conseguir se conectar com o Servidor mesmo não estando em rede local.

8. Implementações Faltantes

Não houve nada mais que gostaríamos de implementar. Obviamente outras funcionalidades para deixar o jogo mais interativo seriam interessantes, mas atingimos nosso objetivo de parodiar o Club Penguin e conseguimos aplicar conceitos de conexão com sockets e trabalhar com threads e sincronismo. Portanto atingimos nossa proposta.

9. Fontes e referências de estudo

LLM:
Google Gemini - Muito utilizado para explicar os conceitos de Sockets, Concorrência e boas práticas de COO. Também foi utilizado para construir a interface gráfica do jogo.

Sites:
tutorialspoint - currentTimeMillis:
https://www.tutorialspoint.com/java/lang/system_currenttimemillis.htm
devmedia - Criando comunicação em Java:
https://www.devmedia.com.br/java-sockets-criando-comunicacoes-em-java/9465
Medium - Criando conexão TCP Client-Server em Java:
https://www.devmedia.com.br/java-sockets-criando-comunicacoes-em-java/9465

Fóruns:
Stack Overflow - Como inicializar JPanel:
https://pt.stackoverflow.com/questions/215526/como-inicializar-um-jpanel-em-branco
Stack Overflow - Java socket API:
https://stackoerflow.com/questions/10240694/java-socket-api-how-to-tell-if-a-connection-has-been-closed

Vídeos:
Java Socket Programming Client Server Messenger:
https://www.youtube.com/watch?v=gchR3DpY-8Q
Java Socket Programming - Multiple Clients Chat:
https://www.youtube.com/watch?v=gLfuZrrfKes
Map and HashMap in Java - Full Tutorial:
https://www.youtube.com/watch?v=H62Jfv1DJlU

10. Diálogos com IA

Durante a criação desta aplicação o Gemini Pro foi utilizado para: em grande parte auxiliar na lógica de uso de threads e listas bloqueantes, como também na implementação da interface gráfica e debug. Segue o log das conversas com a LLM.\
***Tópico 10.1 em PDF separado "Tópico 10.1.pdf" devido ao número de páginas.**